Instrukcja laboratoryjna

XPlanar — TwinCAT 3 (IEC 61131-3)

Spis treści

1 Wstęp 2

1.1 Jak działa XPlanar i jego możliwości . . . 2

1.2 Szafa sterownicza . . . 3

1.3 Środowisko TwinCAT XAE Shell . . . 3

Okno główne . . . 3

Pasek menu i narzędzi . . . 3

Solution Explorer . . . 4

SYSTEM . . . 4

MOTION . . . 5

I/O . . . 6

Ikona TwinCAT w zasobniku systemu . . . 8

1.4 Kod demo . . . 9

Podstawy programowania PLC . . . 9

Program . . . 10

References . . . 11

Linki Inputs i Outputs . . . 12

1.5 Odpalenie projektu . . . 12

Rodzaje pracy: Normal, BasicSimulation, AdvancedSimulation . . . 12

Odpalenie w trybie Normal na PLC . . . 13

Odpalenie w symulacji . . . 13

Symulacja na PLC Beckhoffa . . . 14

Symulacja na zewnętrznym PC . . . 14

1.6 Zadania . . . 14

Zadanie 1 . . . 14

Zadanie 2 . . . 14

2 Konfiguracja oraz włączenie systemu 15

3 Ruch wysterowany 17

4 Tracki i omijanie kolizji 18

5 Dynamiczne trasowanie moverów — stacje 20

1 Wstęp

Forma: prowadzący pokazuje technologię wizualnie (na maszynie), nie na slajdach.

Komentarz / zalecenia

- Student musi otrzymać wiedzę przed przystąpieniem do laboratorium numer 1.
- Wytłumaczenie musi wystąpić **wizualnie** — ilość wiedzy jest zbyt duża, by przedstawić ją na wykładzie, którego slajdy nie odzwierciedlają ani nie uczą realnej pracy przy maszynie.
- Z doświadczenia: po zajęciach z PLC lub jakichkolwiek zajęciach z automatyki przemysłowej studenci nie dochodzą do wyjaśnienia zasady działania ani postępowania z maszyną. Typowy student nawet nie ma pojęcia, czym *dokładnie* jest PLC.
- Wizualne przekazanie wiedzy, co można zrobić z danym sprzętem — choćby na podstawie PLC nie od Beckhoffa, a np. Delty/Siemensa, na realnych przykładach.
- To kluczowe do efektywnego przeprowadzenia laboratoriów, ponieważ systemy Beckhoffa cechują się wysokim poziomem wejścia relatywnie do innych środowisk, jak TIA Portal czy ISPSoft.
- Podkreślić, że na rok 2026 platforma XPlanar Beckhoff jest technologią jeszcze nie w 100% dopracowaną: na razie jest rozwijana i mało jest systemów zaimplementowanych, jednakże jest to rozwiązanie bardzo rzeczowe / obiecujące *(do doprecyzowania sformułowanie)*.

1.1 Jak działa XPlanar i jego możliwości

- jak działa,
- jakie są maksymalne zakresy działania,
- jakie typy i rozmiary,
- możliwości i miejsca zastosowania,
- ograniczenia moverów.

> **[DO UZUPEŁNIENIA]** Opis czynności oraz nazewnictwo bloków, metod i parametrów zgodne z dokumentacją Beckhoff (`Tc3_XPlanarStandard`, `Tc3_XPlanarUtility`).

1.2 Szafa sterownicza

Miejsce na zrzut ekranu

`zdjęcia/01-wstep/szafa-sterownicza/szafa-sterownicza-widok-ogolny.png`

Rysunek 1.1: Szafa sterownicza — widok ogólny (zdjęcie do dodania)

- co znajduje się w szafie,
- jak elementy szafy komunikują się ze sobą i z peryferiami — EtherCAT.

> **[DO UZUPEŁNIENIA]** Opis czynności oraz nazewnictwo bloków, metod i parametrów zgodne z dokumentacją Beckhoff (`Tc3_XPlanarStandard`, `Tc3_XPlanarUtility`).

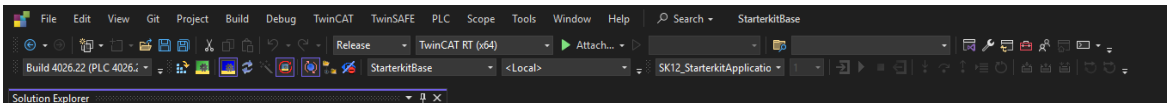
1.3 Środowisko TwinCAT XAE Shell

Projekty TwinCAT 3 tworzymy w środowisku **TcXaeShell 64-bit** (wersja 7.21.11, oparte na Visual Studio 2022) wraz z **TwinCAT XAE 4026.27**.

Okno główne

Okno główne składa się z paska menu i paska narzędzi, Solution Explorera z drzewem projektu, obszaru edytora, w którym otwieramy wybrane elementy projektu, oraz okien pomocniczych (m.in. Error List i Output).

Pasek menu i narzędzi



Rysunek 1.2: Edytor TcXaeShell — pasek menu i pasek narzędzi

Z paska, z którego będziemy realnie korzystać, od lewej:

1. **wybór wersji TwinCAT**, przez którą się komunikujemy, a następnie wersja aktywna (*Active version*),
2. **Activate Configuration** — wgranie konfiguracji projektu na wybrany target,
3. **Run Mode** i **Config Mode** — przełączanie trybu pracy runtime'u (zielona — Run, niebieska — Config),

4. **aktywny projekt** (`StarterkitBase`) — lista rozwijana projektów w rozwiązaniu,
5. **target** (`<Local>`) — system, na którym uruchamiamy projekt.

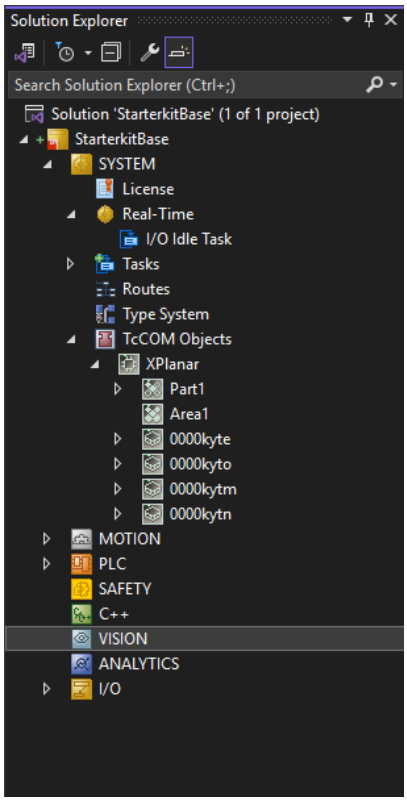
Sterownik zewnętrzny wybieramy po nazwie lub numerze seryjnym; wymaga to wcześniej dodanej trasy komunikacji (*Edit Routes w Zasobniku*).

Solution Explorer

Solution Explorer dzieli projekt na części: SYSTEM, MOTION, PLC, SAFETY, C++, VISION, ANALYTICS i I/O. W laboratorium korzystamy z SYSTEM, MOTION, I/O i PLC.

SYSTEM

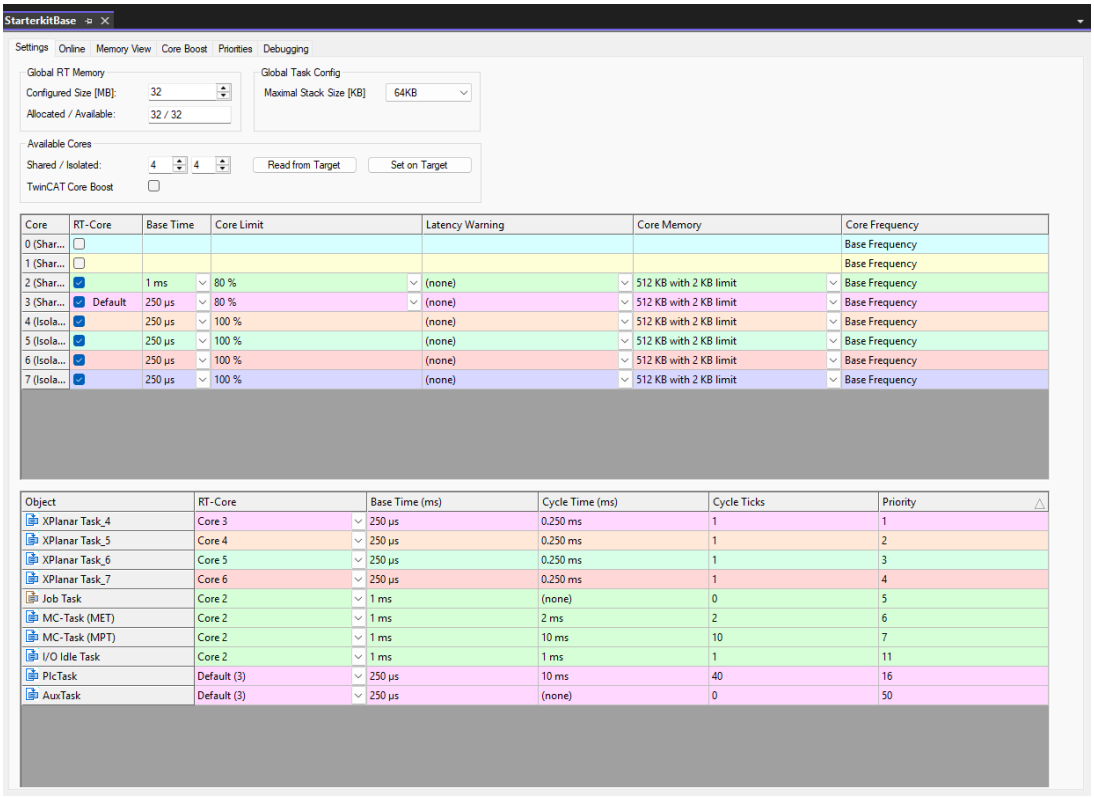
SYSTEM to konfiguracja komputera sterującego, na którym uruchamiamy projekt.



Rysunek 1.3: Solution Explorer — SYSTEM

License — licencje TwinCAT.

Real-Time — w tej zakładce definiujemy rdzenie procesora, które pracują, aby system działał szybko.



Rysunek 1.4: Solution Explorer — SYSTEM, Real-Time (Settings)

Rdzenie *shared* (współdzielone) TwinCAT dzieli z Windows, a *Core Limit* określa, jaką część czasu rdzenia może zająć TwinCAT. Rdzenie *isolated* (izolowane) są przydzielone wyłącznie TwinCAT i Windows z nich nie korzysta. Po zmianie parametrów trzeba zapisać konfigurację (*Set on Target*).

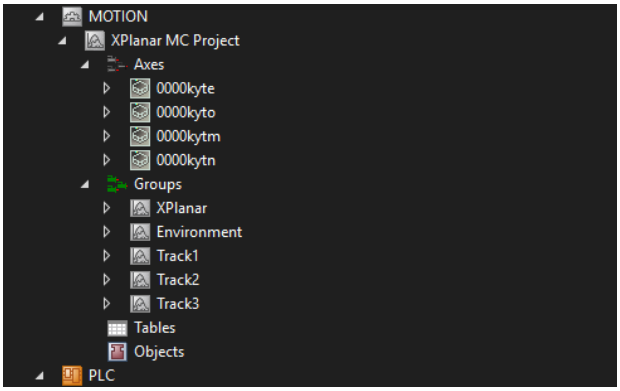
TcCOM Objects — obiekty, w których żyje system XPlanar. Obiekt `XPlanar` zawiera:

- `Part1` — kafelki, po których poruszają się movery,
- `Area1` — obszar ruchu, czyli zdefiniowane środowisko, w którym movery mogą się poruszać,
- movery, nazwane swoim numerem **BTN** (np. `0000kyte`).

BTN (*Beckhoff Traceability Number*) to numer fabryczny movera z jego naklejki. Mover jest tylko magnesem, więc identyfikuje go zderzak (czarna ramka); dzięki BTN system wie, który mover jest którym obiektem w projekcie.

MOTION

`XPlanar MC Project` zawiera obiekty ruchu, które są pośrednikiem między programem PLC a światem rzeczywistym.



Rysunek 1.5: Solution Explorer — MOTION

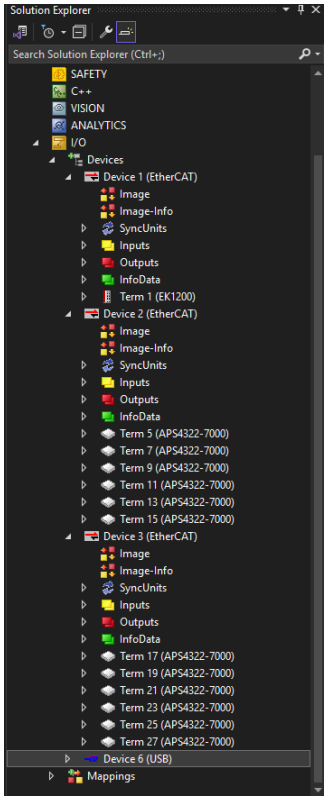
W jego skład wchodzą:

- `Axes` — po jednej osi na każdy mover (to nie są osie silników). Oś jest zlinkowana z obiektem movera w TcCOM oraz ze zmiennymi PLC (`PlcToMc`: komendy, `McToPlc`: stany i pozycja); linki PLC wykonujemy sami (*Change Axis PLC Links*),
- `Groups` — grupa **XPlanar** (sprawdzanie kolizji movery–tracki–środowisko), **Environment** (powierzchnia kafelków, czyli obszar, w którym się poruszamy) oraz **tracki** (*Track1–3*, „ulice” dla moverów).

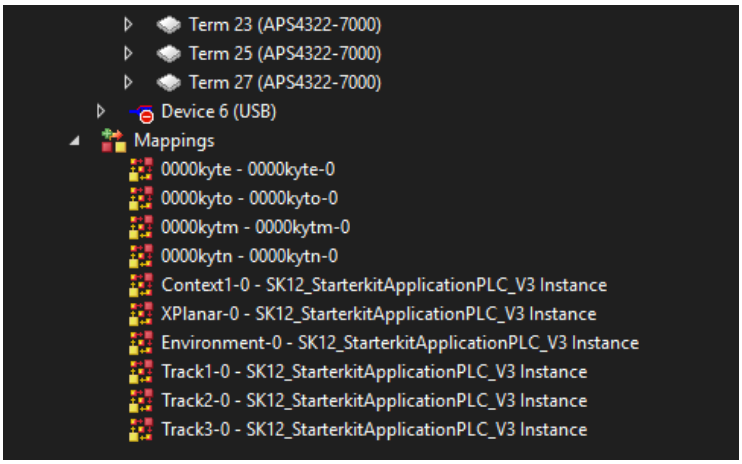
Tylko osie moverów mają odpowiednik w sprzęcie. Grupa, Environment i tracki są czysto programowe, więc linkuje się je tylko z PLC (`MCTOPLC_STD`), aby PLC mogło nimi sterować.

I/O

I/O opisuje fizyczne urządzenia i powiązania między nimi a resztą projektu.



Rysunek 1.6: Solution Explorer — I/O



Rysunek 1.7: Solution Explorer — I/O, Mappings

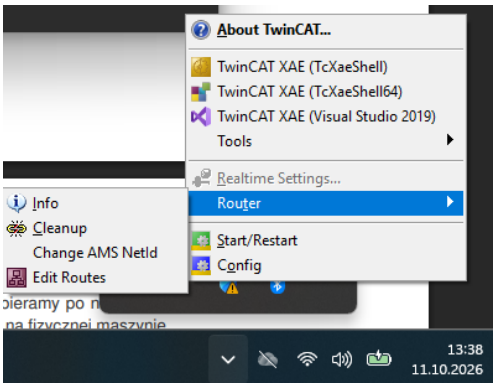
Na zrzutach widać:

- trzy urządzenia EtherCAT: Device 1, Device 2 i Device 3,
- w Device 1 sprzęgacz EK1200,
- w Device 2 terminale APS4322-7000 o numerach 5, 7, 9, 11, 13 i 15, czyli kafelki XPlanara,
- w Device 3 terminale APS4322-7000 o numerach 17, 19, 21, 23, 25 i 27, również kafelki,
- w każdym urządzeniu EtherCAT elementy wspólne: Image, Image-Info, SyncUnits, Inputs, Outputs i InfoData,
- urządzenie USB: Device 6,

- Mappings, czyli wszystkie linki w jednym miejscu: obiekty moverów 0000kyte, 0000kyto, 0000kytm i 0000kytn powiązane ze swoimi osiami, a także Context, XPlanar, Environment i Track1–3 powiązane z instancją projektu PLC.

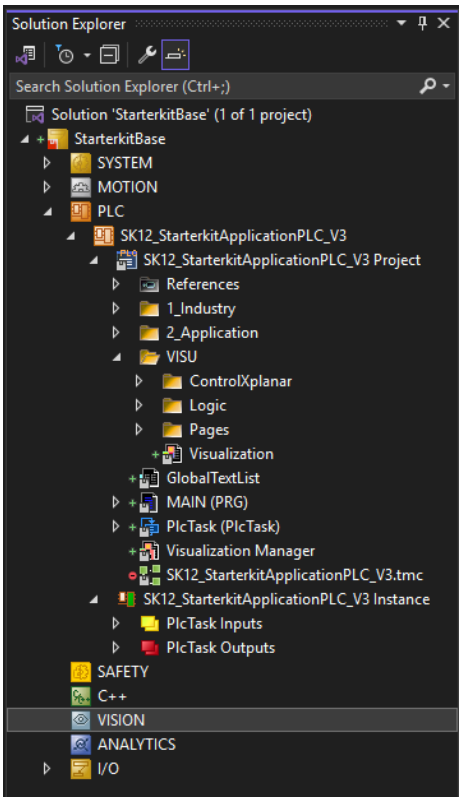
Ikona TwinCAT w zasobniku systemu

Menu ikony TwinCAT w zasobniku systemu pozwala m.in. uruchomić środowisko (TcXaeShell 32/64-bit, Visual Studio), uruchomić lub zrestartować runtime (`Start/Restart`), przełączyć go w tryb Config (`Config`) oraz otworzyć ustawienia routera ADS (`Router`: `Info`, `Cleanup`, `Change AMS NetId`, `Edit Routes`). Stan runtime'u widać po kolorze ikony (niebieska — Config, zielona — Run); szczegóły: Information bar w Infosys.



Rysunek 1.8: Menu ikony TwinCAT w zasobniku systemu

1.4 Kod demo



Rysunek 1.9: Solution Explorer — PLC

W tym projekcie skupiamy się tylko na folderze `2_Application`.

Podstawy programowania PLC

Językiem programowania jest **ST** (*Structured Text*).

Zasady. Ze względu na to, że jest to PLC, nasz program wykonuje się **cyklicznie**, podobnie jak w ladderze czy w języku blokowym: zadanie `PlcTask` wywołuje program główny w każdym cyklu (tu co 10 ms). Dlatego kod nie może czekać w pętli; długie czynności rozkładamy na kroki maszyny stanów (`CASE`), wykonywane w kolejnych cyklach. Poza tym:

- zmienną deklarujemy przed użyciem, w bloku `VAR ... END_VAR`, z typem i opcjonalną wartością początkową,
- instrukcję kończymy średnikiem; przypisanie to `:=`, porównanie to `=`.

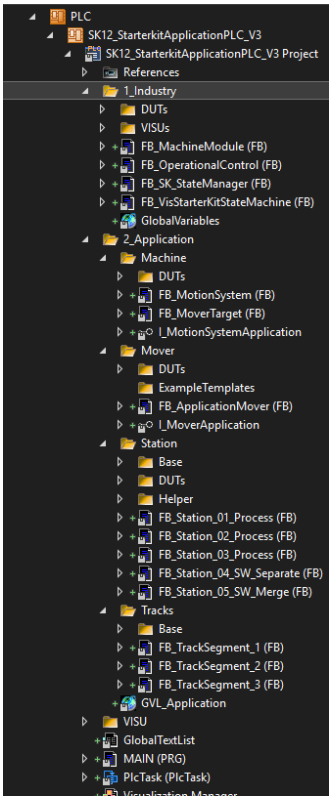
Jednostki organizacyjne (POU). Program składa się z jednostek organizacyjnych (*Program Organization Unit*), które dodajemy w Solution Explorer:

- **Program** — jednostka wywoływana bezpośrednio przez zadanie; jest jedna i zachowuje stan między cyklami,
- **Function Block** — jednostka z własną pamięcią: tworzymy jej instancje, a każda pamięta swój stan. Może mieć metody i właściwości, dziedziczyć po innym bloku i implementować interfejsy,
- **Function** — jednostka bez pamięci, która dla tych samych argumentów zwraca ten sam wynik,

- **Method** — funkcja należąca do bloku funkcyjnego lub programu, wywoływana na jego instancji, z dostępem do jego zmiennych,
- **Property** (właściwość) — z zewnątrz działa jak zmienna bloku, ale odczyt i zapis wykonuje kod (*Get* i *Set*),
- **Action** — dodatkowy fragment kodu bloku lub programu, bez własnych argumentów, pracujący na jego zmiennych,
- **Interface** — lista metod i właściwości bez kodu; blok, który go implementuje, musi je zawierać, dzięki czemu różne bloki można traktować jednakowo,
- **DUT** (*Data Unit Type*) — definicja własnego typu danych: struktury, wyliczenia lub aliasu,
- **GVL** (*Global Variable List*) — lista zmiennych i stałych globalnych, dostępnych w całym projekcie.

Dobre praktyki nazewnictwa. Nazwy zmiennych mają prefiks oznaczający typ: `b` — `BOOL`, `n` — liczba całkowita, `f` — zmiennoprzecinkowa, `s` — `STRING`, `fb` — instancja bloku funkcyjnego, `ip` — interfejs. Typy i listy mają prefiks rodzaju: `FB_`, `ST_` (struktura), `E_` (wyliczenie), `I_` (interfejs), `GVL_`; stałe globalne zaczynamy od `gc_`.

Program

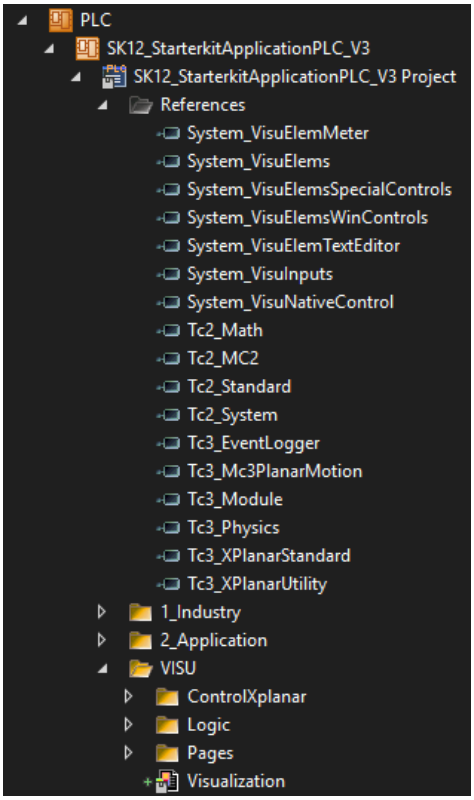


Rysunek 1.10: Solution Explorer — PLC, kod aplikacji

- `Machine` — `FB_MotionSystem` to główna maszyna stanów systemu: zatrzymanie, reset, przygotowanie i praca. W przygotowaniu sprawdza zasilanie kafelków, wykrywa i identyfikuje movery, określa ich orientację, włącza je i wprowadza na tory. `FB_MoverTarget` wysyła movery do zadanych pozycji,
- `Mover` — `FB_ApplicationMover` to obiekt aplikacyjny jednego movera: w każdym cyklu wywołuje standardowy obiekt movera i udostępnia jego pozycję wizualizacji,

- `Station` — `FB_StationBase` to wspólna baza dla pięciu stacji. Każda stacja jest maszyną stanów z metodami `Init`, `Clear` i `Process`: czeka na movera, wysyła go do pozycji obróbki, wykonuje swój proces, wybiera następną stację i odsyła movera po torze. Nazwy stacji 4 i 5 (`SW_Separate`, `SW_Merge`) wskazują na rozdzielanie i łączenie torów,
- `Tracks` — `FB_TrackBase` i trzy segmenty torów, które po nim dziedziczą; metoda `Initialize` buduje tablicę punktów toru,
- `GVL_Application` — stałe aplikacji: 4 movery, 3 tory i 5 stacji.

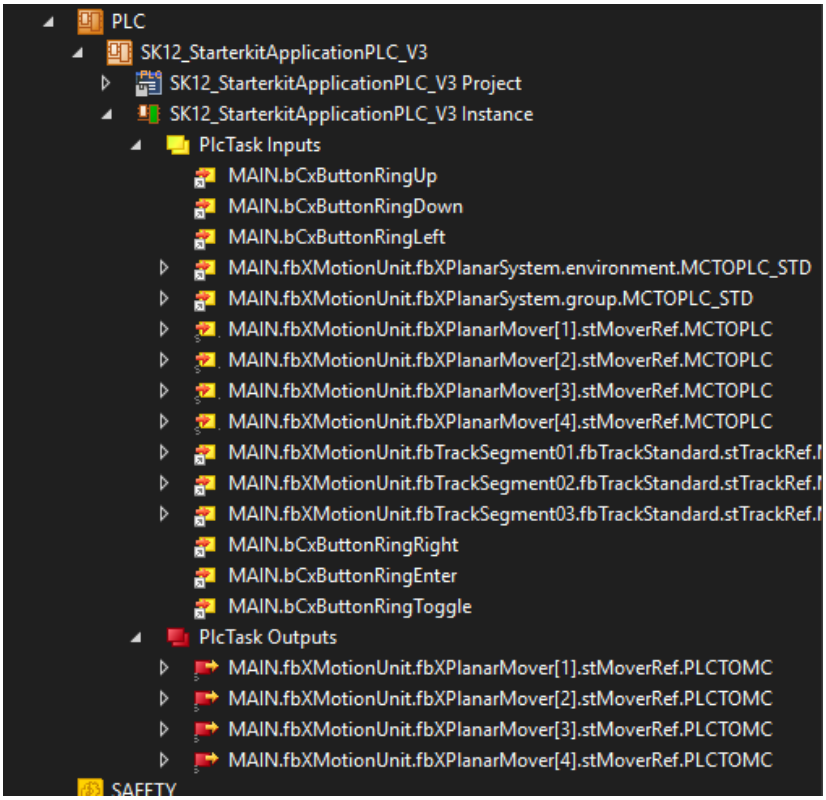
References



Rysunek 1.11: Solution Explorer — PLC, References

References to biblioteki dołączone do projektu: gotowe bloki i typy, z których korzysta program. Używamy między innymi `Tc3_XPlanarStandard`, która nie jest ogólnie udostępniana przez Beckhoff. Dzięki niej pisanie programu jest wyższego poziomu: zamiast niskopoziomowych komend ruchu używamy gotowych obiektów systemu i movera, np. wywołania `std.cycle()`.

Linki Inputs i Outputs



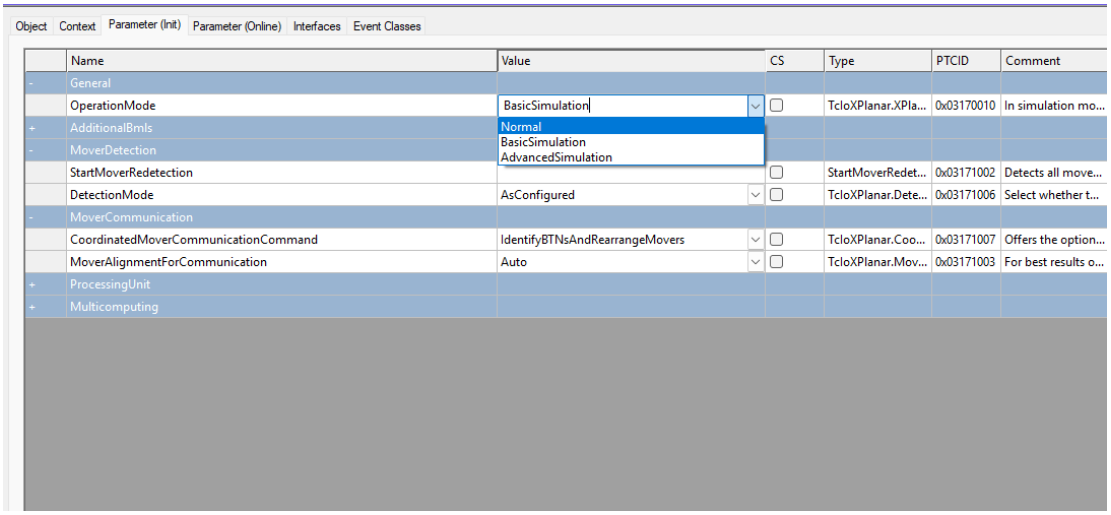
Rysunek 1.12: Solution Explorer — instancja PLC (Inputs, Outputs)

Instancja projektu PLC ma dwie listy zmiennych wymienianych z resztą systemu w każdym cyklu zadania. **PlcTask Inputs** to dane wchodzące do PLC: przyciski pierścienia CX, a także statusy z MOTION (`MCTOPLC`) dla Environment, grupy, czterech moverów i trzech torów. **PlcTask Outputs** to dane wychodzące z PLC: komendy dla czterech moverów (`PLCTOMC`). **Link** to połączenie zmiennej PLC ze zmienną drugiej strony, np. osi w MOTION; dzięki niemu wartości są kopiowane automatycznie w każdym cyklu, a PLC może sterować moverami. Zmienne przyciskowe w programie są zadeklarowane jako wejścia (`AT %I*`), czyli bez stałego adresu, który nadaje dopiero link.

1.5 Odpalenie projektu

Rodzaje pracy: Normal, BasicSimulation, AdvancedSimulation

Tryb ustawiamy w obiekcie XPlanar: `SYSTEM` → `TcCOM Objects` → `XPlanar` → `Parameter (Init)` → `OperationMode`.



Rysunek 1.13: Obiekt XPlanar — Parameter (Init), wybór OperationMode

Tryb	Sprzęt	Czas rzeczywisty	Obliczenia
`Normal`	wymagany	wymagany	rzeczywisty system
`BasicSimulation`	praca na komputerze	nie musi być spełniony, można symulwoać na jednym rdzeniu	symulowany ruch moverów, bez obliczania prądów i sprzężenia zwrotnego
`Advanced-Simulation`	?	wymagany; układ zadań jak w rzeczywistej konfiguracji (czas cyklu, rozłożenie obciążenia na rdzenie)	aktywne obliczanie prądów i sprzężenia zwrotnego, bliżej trybu `Normal`

Odpalenie w trybie Normal na PLC

Tryb `Normal` to praca z rzeczywistym sprzętem na sterowniku XPlanara.

1. Otwieramy rozwiązanie (aktualnie: `StarterkitBase`) w TcXaeShell.
2. Wybieramy target: sterownik XPlanara (np. `CX-94E797`); wymaga to dodanej wcześniej trasy komunikacji.
3. Sprawdzamy, czy `OperationMode` to `Normal`.
4. Sprawdzamy, urządzenia I/O są enablowane.
5. Upewniamy się, że movery stoją na kafelkach.
6. Wykonujemy *Activate Configuration*, logujemy się do PLC i uruchamiamy program.
7. Maszynę obsługujemy z wizualizacji: Start, Stop, wprowadzenie w stan Execute.

Odpalenie w symulacji

W symulacji projekt działa bez maszyny: symulowany jest sprzęt XPlanar (kafelki i movery), a program PLC wykonuje się normalnie. Wspólne kroki:

1. w obiekcie XPlanar ustawiamy `OperationMode` na `BasicSimulation` lub `AdvancedSimulation`,
2. urządzenia I/O ustawiamy na disable albo na simulation mode.
3. wykonujemy *Activate Configuration*, logujemy się do PLC i uruchamiamy program,

4. zmiany w symulacji obserwujemy na dwa sposoby:
    - *TwinCAT* → *MC3-XPlanar Environment View*,
    - *TwinCAT* → *XPlanar* → *Configurator* → *Live View*; po zmianach w projekcie klikamy *Reload*,

Symulacja na PLC Beckhoffa

Jako target wybieramy sterownik. Runtime jest na nim już zainstalowany i skonfigurowany pod czas rzeczywisty, więc kroki dotyczące wirtualizacji i pakietów z następnego podrozdziału nie mają zastosowania. `BasicSimulation` nie wymaga czasu rzeczywistego, a `AdvancedSimulation` wymaga rzeczywistego układu zadań, który sterownik zapewnia. Przy niezgodności ustawień Real-Time wykonujemy *Read from Target*.

Symulacja na zewnętrznym PC

Jako target wybieramy `<Local>`. Komputer musi spełniać wymagania:

- zainstalwoanwego środowiska TwinCAT w pdoanej konfigruacji. TODO DODOAĆ KONFIURACJE WYŁAUMCZYĆ PBORANIE NA KOMPUTER
- wyłączona wirtualizacja: Hyper-V, WSL, VBS (Memory integrity) i Windows Hello Enhanced Sign-in; zmiana wymaga uprawnień administratora i restartu,
- biblioteka `Tc3_XPlanarStandard` w wersji, której wymaga projekt (References),
- ustawienia Real-Time dopasowane do komputera: *Read from Target*, potem przypisanie zadań do istniejących rdzeni; dla `BasicSimulation` wystarczy jeden rdzeń, dla `AdvancedSimulation` układ zadań jak w rzeczywistej konfiguracji.

1.6 Zadania

Zadanie 1

Uruchomienie projektu na PLC lub przez własny komputer w trybie Normal, odpalić z visualizacji ruch moverów

Zadanie 2

Odpalenie Symualcji w trybie BasicSimulation na PLC lub przez własny komputer, odpalić z visualizacji ruch moverów oraz sprawdic ruch

Dokumentacja Beckhoff (Infosys)

- TF5430 TwinCAT 3 Planar Motion — Introduction
- Installing TwinCAT 3 Engineering and Runtime
- Information bar (stan systemu TwinCAT)
- Your first TwinCAT 3 PLC project
- Core Management (Real-Time: rdzenie, Base Time)

2 Konfiguracja oraz włączenie systemu

Ćwiczenia: ***inicjalizacja systemu***, ***lewitacja***

Backup

- **STWORZENIE BACKUP** — student musi umieć tworzyć backup.
- Trzeba nauczyć dobrych praktyk: zawsze przed wejściem do środowiska produkcyjnego wykonujemy backup.

Backup systemu na sterowniku Beckhoff (TwinCAT 3, Windows) — skrótowo

Backup składa się z trzech części; zawsze robimy je razem:

1. **Archiwum projektu (źródła)** — w drzewie projektu prawy klik na projekcie PLC → `Save <nazwa projektu> as Archive...`. Wynikiem jest plik archiwum (u nas `.tnzip`), z którego projekt można odtworzyć na innym komputerze.
2. **Boot project** — projekt na sterowniku musi odpowiadać projektowi w środowisku. Zapisz zmiany (czerwona dyskietka przy obiektach oznacza niezapisane zmiany), a następnie na węźle PLC wybierz `Activate Boot Project`, zaloguj się i uruchom PLC.
3. **Kopia folderu Boot ze sterownika** (przez RDP lub udział sieciowy): w TwinCAT 3.1.4026 i nowszych `C:\ProgramData\Beckhoff\TwinCAT\3.1\Boot\`. Folder `Plc` zawiera dane boot aplikacji PLC, a opcjonalnie `CurrentConfig` — źródła projektu, jeśli są przechowywane na sterowniku.

> **Note**
> Zmienne `PERSISTENT` zapisują się w pliku `.bootdata` w folderze Boot (poprzednią wersję system zmienia na `.bootdata-old`). Aby mieć w backupie aktualne wartości, wywołaj przed kopiowaniem `FB_WritePersistentData` z biblioteki `Tc2_Utilities`.

Przywracanie: rozpakuj archiwum projektu na komputerze, zawartość skopiowanego folderu `Plc` wstaw do folderu `Boot\Plc` na sterowniku (zastępuje istniejącą), po czym zrestartuj TwinCAT.

> **Note**
> Osobny produkt **TwinCAT Backup** (usługa `TcBackupServer`, archiwa `*.tbs`, profile z harmonogramem) wg Infosys należy do dokumentacji **TS1150 | TwinCAT 2 Backup**, więc dla TwinCAT 3 nie jest tu opisany. Źródła: Infosys Beckhoff, rozdziały *Performing a PLC update*, *Transferring a PLC project*, *PLC Project Settings*.

Pakiety i biblioteki

- Sprawdzenie i pokazanie działania Package Managera po stronie Beckhoffa.
- Przedstawienie bibliotek.

Projekt i sprzęt

- Utworzenie projektu, zdefiniowanie obszaru roboczego XPlanar.

- Student musi wiedzieć, jak wygląda tile.
- Wytłumaczenie po krótce, jak działa EtherCAT — **max 6 tile'i na jeden EtherCAT**.
- Wsadzenie moverów, odczytanie BTN.

Odpalenie moverów

- Movery można odpalić tylko z przechyleniem do 10 stopni w osi C.
- Wspomnieć o ograniczeniu: nie znamy dokładnie, w jakiej orientacji jest mover *(odnośnik do strony Beckhoffa; w dalszej części kursu nie będzie to wspominane)*.
- **Z moverUI:**
  - pokazanie, jak działa JOG,
  - pokazanie punktów rotacji, zmiany Z i innych możliwości movera *(do sprawdzenia: czy na dużych moverach są punkty rotacji)*,
  - pokazanie ruchu w osi Z *(można też spróbować A i B, ale nie wiem po co)*.
- Na końcu: pokazanie, że movery można enable'ować przez kod — wytłumaczenie cyklu.

Zadanie *(propozycja)*: uruchom system w symulacji (`BasicSimulation`), zaloguj się do PLC, włącz movery z kodu (`EnableMovers`), unieś jeden mover w Z na 5 mm i opuść go z powrotem; podglądaj pozycję w `P_ActPosition` i w Configuratorze (Live View).

- **Lewitacja:** funkcje do kontroli w osi Z (`MoveZ`, zakres w konfiguracji osi: od -1 do 10 mm) *(przeniesione z zagadnienia 3)*.
- Metoda do usuwania i dodawania moverów + wytłumaczenie, po co jest potrzebna *(przeniesione z zagadnienia 3)*.
- Pokazanie, jak to działa dla **2 wariantów bibliotek**:
  - `Tc3_XPlanarUtility` — ogólnie dostępna,
  - `Tc3_XPlanarStandard` — poza repozytorium Beckhoffa, udostępniana na prośbę.

Materiały do przekazania

- Do tego etapu: GitHub, nagranie oraz docs w PDF.

Dokumentacja Beckhoff (Infosys)

- Installing TwinCAT 3 Engineering and Runtime
- Information bar (stan systemu TwinCAT)
- Command Activate configuration
- Activating simulation mode
- Parameter (Init) — OperationMode
- TwinCAT 3 XPlanar Utility

3 Ruch wysterowany

Ćwiczenie: ***ruch w 6 stopniach swobody (6 DoF)***

Start

- Z GitHuba, z brancha 3, pobrać działający projekt albo kontynuować kolejny.

Cel dydaktyczny

- Student powinien rozumieć, **dlaczego mamy cykl**.
- Pokazanie studentowi, jak ruszyć jednym moverem.
- Student musi umieć dodać w pętli `FOR` kolejne movery — zrozumieć, że tak się je deklaruje i że są aktualizowane z cyklu.

Ruch zadany w 6 DoF — free move

Każda oś ma osobną komendę na obiekcie movera (`FB_XPlanarMoverStandard`):

Oś	Komenda	Uwagi
X, Y (+C)	`MoveToPosition`	ruch po płaszczyźnie; zadaje X/Y/C naraz
C (obrót)	`MoveC`	tylko w punktach obrotu (co 120 mm); `P_AtRotationPoint`
Z (wysokość)	`MoveZ`	zakres od -1 do 10 mm
A (przechył wokół X)	`MoveA`	zakres $\pm 6^\circ$
B (przechył wokół Y)	`MoveB`	zakres $\pm 6^\circ$

- **A i B nie są w `MoveToPosition`** — zadaje się je osobnymi komendami `MoveA` / `MoveB` (parametry: pozycja docelowa w stopniach, prędkość, przyspieszenie, opóźnienie, jerk), tak samo jak `MoveZ`.
- Zastosowanie przechyłu: np. kompensacja przechyłu przy płynach, ustawienie narzędzia pod kątem.

Zadanie

Wykonaj ruch 4 moverów w tył i w przód w osi Y.

- Punkty startowe: `Y = 360`; `X` pierwszego = `120`, ostatniego = `840`.
- Po tym etapie student powinien rozumieć działanie cyklu, jak dodać movery itd.
- *(Rozszerzenie 6 DoF — do decyzji: w punktach końcowych podnieść movery w Z i przechylić w A/B.)*

Dokumentacja Beckhoff (Infosys)

- Limits and options of the motion commands
- Example: Activating a Planar part position and moving a Planar mover
- Example: Creating and moving a Planar mover with External Setpoint Generation

4 Tracki i omijanie kolizji

*Ćwiczenie: **omijanie kolizji** (tracki jako podstawa)*

Tracki

Start

- Pobieramy z GitHuba *(do sprawdzenia: czy da się to dodać)* wersję kodu, w której jest dodany cały blok Motion, Mover Preparation itd.
- Student powinien się zaznajomić, co dokładnie dają te bloki.

Koncept tracków

- Pokazanie konceptu tracków poprzez dokumentację — jak go dodać.
- Przedstawienie ograniczeń (produkt nie jest jeszcze w 100% dopracowany).
- Skupienie się na razie na **liniach**.
- Zlinkowanie tracków.
- **Group** — deklarujemy już tutaj, bo tracki zawsze dodaje się do grupy (`EnableGroup` → `AddEnvironmentToGroup` → `EnableTracks` → `AddTracksToGroup` → `EnableMovers` → `AddMoversToGroup`). Szczegóły działania grupy — poniżej, w części o kolizjach.

Pokaz w UI

- Jak wejść na track.
- Dlaczego **keep orientation** jest ważne.
- JOG po tracku.
- Metoda **recovery**, która pozwala dołączyć do tracku *(powiedzieć o ograniczeniach — czy jest w stanie bezproblemowo nie uderzyć w inne movery)*.

Omijanie kolizji — Group

- **Group** — jak działa; odesłanie do linku Beckhoffa.
- Environment jako granica obszaru ruchu.
- Pokazanie na zasadzie kolejki *(do sprawdzenia: czy nie trzeba kolejnego brancha na GitHubie, by pokazać kolejkowanie)* — `gap`, `P_IsQueuing`.
- Błąd **33158** (Collision Avoidance odrzuciło ruch) i odczyt `stInfo` — który obiekt zablokował.
- Strefa wspólna — materiał z rozdziału 08 pracy.

Zadania

Zadanie 1 — jeden track, dwa movery:

- Pokazanie, że na tracka można wejść **ręcznie** (z UI).

- Zapisanie kodu dla jednego tracka (budowa geometrii, linkowanie, dodanie do Group), dołączenie do niego 2 moverów i wysterowanie jazdy w kółko — jak w docsach Beckhoffa.

Zadanie 2 — dwa tracki z punktem przecięcia:

- 2 movery jadące na **oddzielnych trackach**, które przechodzą przez jeden punkt przecięcia (łatwe do zrobienia).
- Obserwacja zatrzymania, kolejkowania i wznowienia przy kolizji w punkcie przecięcia.

Dokumentacja Beckhoff (Infosys)

- Planar track operation modes
- Planar environment
- Collision avoidance between Planar movers and Planar tracks
- Example: Creating and moving Planar movers with group
- Example: Creating and moving Planar movers with track and group
- Example: Following a Planar mover through a Track Network

5 Dynamiczne trasowanie moverów — stacje

Ćwiczenie: ***dynamiczne trasowanie moverów*** *(najtrudniejsze)*

- Stacje jako punkty decyzyjne — mover po obsłudze dostaje kolejny cel w zależności od stanu systemu.
- Wzorzec ze starter kitu: `FB_Station_01..03` (stacje procesowe, wybór następnej stacji), `FB_Station_04_SW_Separate` (zwrotnica 1→2), `FB_Station_05_SW_Merge` (zwrotnica 2→1 z priorytetem).
- Listy kolejek stacji (`FB_List`), `P_ProcessRequest` / `P_ProcessClearance`.
- `FB_MoverTarget` — przydział najbliższego movera do pozycji.

Zadanie *(propozycja)*: na torach z zagadnienia 3 dodać 2 stacje i zwrotnicę; movery cyklicznie trafiają do wolnej stacji.

Dokumentacja Beckhoff (Infosys)

- TF5430 TwinCAT 3 Planar Motion — Introduction
- Example: Following a Planar mover through a Track Network